Finite-space queue topology synthesis

Sparse optimal designs, heterogeneous service, infrastructure budgets, and sensitivity

Research working note

27 October 2026

Abstract

We extend a deterministic, finite-population queue-clearance model in five directions. For the original row family, we determine exactly how many counters must be used by an optimal design, prove sharp portal counts in several parameter regimes, and obtain a one-portal gap elsewhere when the source admits at least two people per tick. A separate finite candidate-graph formulation chooses waiting cells, corridors, counters, and optional discrete counter placements, with heterogeneous processing durations and construction and area constraints. We compute exact finite-instance frontiers and examine receiving rules, corridor capacity, decentralized control, and random durations. The algebraic clearance formula is situated within established max-plus and production-system theory. The proposed contribution is the sparse finite-horizon attainment result under the stated physical constraints, not a new general theory of queues or a universal optimal floorplan.

Status of claims. The structural theorems below have explicit mathematical proofs and constructive schedules. The finite-instance optima are certified by integer optimization with zero reported gap, with separate integer replay of every feasible trace. Infeasibility certificates rely on the numerical MILP solver; they are not independently checked rational proof objects. The novelty claim is deliberately narrower than a claim of priority: the reviewed literature does not establish that these exact sparse constructions have appeared before, but this review is not evidence that they have never appeared.

Main deliverables. General portal bounds; a heterogeneous finite-graph solver; a complete illustrative budget frontier; spatial topology and discrete placement examples; exact enumeration of a two-point service-time distribution; and source code, data, schedule witnesses, and reproduction instructions.

Contents

1 What is being optimized? 3

1.1 The original row family 3

2 The exact uniform-service clearance benchmark 4

2.1 The max-plus relation, and what it does not prove by itself 4

3 General minimum-connection theorems 4

3.1 When the bounds become exact 5

4 Heterogeneous service and finite-graph synthesis 7

4.1 What remains simple without movement 7

4.2 A safe bound with movement 7

4.3 The general model and its exactness 8

4.4 A discovered spatial topology 9

5 A physical construction and area budget 10

6 Sensitivity and operating policies 11

6.1 Receiving-space reuse changes the theorem 11

6.2 Corridor capacity and merge storage interact 11

6.3 Static versus dynamically opened connections 12

6.4 Random durations and decentralized policies 12

7 Position relative to established research 14

8 What this can contribute to networks 15

9 Verification, reproducibility, and remaining scope 16

1 What is being optimized?

The objective is

$$T^*(B, A) = \min_{G \in \mathcal{G}(B,A)} \min_{\pi \in \Pi(G)} \max_{1 \le i \le N} t_i^{\text{exit}},$$

where every person must complete exactly one service and then reach the absorbing exit. The layout G and the admission, routing, service-start, and release policy π are jointly chosen. Here B and A denote construction and area budgets. Cells have finite storage, interfaces have finite movement capacities, and downstream saturation can prevent a completed job from leaving its counter and then block upstream cells.

Population and service times alone do not specify $\mathcal{G}$. A corridor-shaped footprint and an open hall can admit different optimal designs with the same N and p_j. Our exact global statements therefore always name the admissible design space. The row-family results optimize all subsets of specified interfaces. The generic solver optimizes all subgraphs of a supplied spatial candidate graph, and can also select among supplied counter-placement alternatives. A continuous, unrestricted floorplan optimization is a different problem.

1.1 The original row family

There are m rows. Each contains two approach cells A_j, B_j, a one-person counter C_j, and a one-person departure cell E_j. A finite source X_0 contains all N people initially. Departure cells lead to a common merge M, then to the absorbing exit X_1.

The optional physical portals are $X_0 \to A_j$, $A_j \to B_j$, and the adjacent lateral interfaces in the A and B columns. Each lateral interface permits either direction but has a combined movement capacity of one per tick. There are $4m - 2$ candidate portals. Service interfaces and links $E_j \to M \to X_1$ are fixed and are excluded from this portal count.

Approach cells have storage two. Each has sending and receiving capacity one. The source can admit up to Q_0 people per tick, with $Q_0 \ge 1$. Its storage K_0 must satisfy $K_0 \ge N$ because everyone is initially present there. If $K_0 < N$, this initial condition is infeasible; an external arrival or staging process would be an additional model input. Source storage and source admission capacity are distinct parameters.

Movement from one ordinary cell to another takes one tick. Service begins when a person is removed from B_j at integer time t. Service lasts p full ticks. A completed person can be released at time $t + p$ or later, reaches E_j one tick after release, and can eventually move through M to X_1. A release and a new service start at the same counter may occur at the same time. This gives earliest possible exit time

$$D = p + 5.$$

The strict synchronous convention measures receiving space before any movement in that tick:

$$O_v(t) \le \min\{n_v(t), Q_v\}, \tag{1}$$

$$I_v(t) \le \min\{K_v - n_v(t), Q_v\}, \tag{2}$$

$$n_v(t+1) = n_v(t) + I_v(t) - O_v(t). \tag{3}$$

A full cell cannot receive during the tick in which it sends. All results in Sections 2 and 3 use this convention, unit corridor rates, and the storage sizes just stated. The integer ticks are model units, not empirically calibrated seconds.

2 The exact uniform-service clearance benchmark

Define

$$\delta = \begin{cases} 2, & K_M = 1, \\ 1, & K_M \geq 2, \end{cases} \qquad b = \max(p, 2), \qquad P_m = \max(b, m\delta).$$

The one-place departure cells force consecutive releases from a given counter to be at least two ticks apart. Processing also forces separation by at least p. The merge requires output separation at least δ. We use $(x)_+ = \max\{x, 0\}$.

Theorem 1 (Uniform-service optimum)**.** *For the row family, $N, m, p \geq 1$, $K_M \geq 1$, $K_0 \geq N$, and $Q_0 \geq 1$, the globally minimum clearance time is*

$$\begin{aligned} T_m^* &= p + 5 + P_m \left\lfloor \frac{N-1}{m} \right\rfloor + \delta((N-1) \bmod m) \qquad (4) \\ &= D + \delta(N-1) + (b - m\delta)_+ \left\lfloor \frac{N-1}{m} \right\rfloor. \end{aligned}$$

The minimum is over every admissible portal subset and every feasible controlled schedule.

Proof. Write $N - 1 = qm + r$ with $0 \leq r < m$. No exit occurs before D, so merge spacing gives $T \geq D + \delta(N-1)$. Each counter's $(q+1)$st exiting person cannot arrive before $D + qb$, because its releases are at least b apart. Before that time at most qm people can have exited. The remaining $r + 1$ outputs require at least $r\delta$ more ticks. Hence $T \geq D + qb + r\delta$. Taking the larger bound gives (4).

For attainment, open all direct approach pairs and start service in row $j + 1$ at $2 + kP_m + j\delta$, for $j = 0, \ldots, m - 1$, stopping after N jobs. Send each job from the source two ticks before its service start, then from A to B one tick before. Release after p ticks; move from E one tick after release and from M another tick later. Consecutive source admissions are at least one tick apart. Each counter's service starts and releases are $P_m \geq b$ apart. Successive merge arrivals are at least δ apart, including across rounds. Finite-cell sending, receiving, and storage constraints are therefore satisfied. The final exit has exactly the time in (4). □

2.1 The max-plus relation, and what it does not prove by itself

The optimal exit sequence is represented by

$$C_i = D + (i-1)\delta \quad (1 \leq i \leq m), \qquad C_n = \max\{C_{n-1} + \delta,\ C_{n-m} + b\} \quad (n > m).$$

Solving this recurrence gives (4). Its long-run cycle time is $\max(\delta, b/m)$, the usual maximum-cycle-mean form. This recurrence represents the optimum; it is not asserted to govern every feasible delayed or backlogged schedule. The algebra alone does not show that a sparse, spatially feasible layout attains it. The constructions and connectivity lower bounds below supply that additional result.

3 General minimum-connection theorems

Let L^* be the minimum number of constructed optional portals among layouts attaining T_m^*. Define

$$r_* = \min\left\{N, m, \left\lceil \frac{b}{\delta} \right\rceil\right\}.$$

Theorem 2 (Exactly how many counters are necessary)**.** *r_* is the minimum number of counters that serve at least one person in a time-optimal layout. In particular,*

$$r_* + 1 \le L^* \le 2r_*.$$

Proof. With only k active counters, the preceding lower-bound proof gives $T \ge T_k^*$. Direct access to any k rows attains this bound. The additional term beyond the merge bound is

$$F(k) = (b - k\delta)_+ \left\lfloor \frac{N-1}{k} \right\rfloor.$$

If $r_* < m$, either $r_* = N$ or $r_* = \lceil b/\delta \rceil$, so $F(r_*) = 0$. For any $k < r_*$ both factors are positive. If $r_* = m$ and $F(m) > 0$, reducing k strictly increases the first factor and does not decrease the positive second factor. The boundary case $F(m) = 0$ follows as before. Thus $T_k^* > T_m^*$ for every $k < r_*$, while $T_{r_*}^* = T_m^*$.

Every used B cell is connected to the source in the undirected graph of used approach portals. This graph contains the source, at least r_* distinct B cells, and at least one A cell. A connected graph on these vertices needs at least $r_* + 1$ edges. Opening r_* direct entry/forward pairs gives the upper bound. □

Theorem 3 (One-place merge: exact sparsity for all parameters)**.** *If $K_M = 1$, then for every $Q_0 \ge 1$,*

$$\boxed{L^* = r_* + 1.}$$

Proof. Use one entry $X_0 \to A_1$, one forward link $A_1 \to B_1$, and a chain of $r_* - 1$ adjacent B portals. Put $r = r_*$ and $P = \max(b, 2r)$. For round k and counter $h + 1$, $0 \le h < r$, dispatch at $kP + h$, use the unique $2 + h$-step approach, and start service at $2 + kP + 2h$. The jobs form a pipeline through the chain. At most one old job occupies any approach cell at a time; receiving while sending uses its second storage place. Counter releases are $P \ge b$ apart and successive merge arrivals are at least two ticks apart. Truncate after N jobs. This gives $T_r^* = T_m^*$ with $r + 1$ portals, matching the connectivity bound. □

Theorem 4 (Two admission opportunities leave at most a one-portal gap)**.** *If $K_M \ge 2$, $Q_0 \ge 2$, and $r_* \ge 2$, then*

$$\boxed{r_* + 1 \le L^* \le r_* + 2.}$$

If $r_* = 1$, then $L^* = 2$.

Proof. Write $r = r_*$, $\ell = \lceil r/2 \rceil$ and $P = \max(b, r)$. Split the r consecutive B cells into two chains rooted at adjacent central rows ℓ and $\ell + 1$. Connect each root directly to the source through its own A cell; open only the outward B-chain interfaces. There are $4 + (r - 2) = r + 2$ portals.

At left-chain distance h, start service at $2 + kP + 2h$ and dispatch at $kP + h$. At right-chain distance h, start at $3 + kP + 2h$ and dispatch at $kP + h + 1$. The two chains provide all service-start phases $0, 1, \ldots, r - 1$. Source use is at most two per tick. Each chain has the same finite-buffer pipeline property as in the preceding proof; $P \ge r$ prevents successive admission batches from overlapping in an approach cell. Departure-cell release spacing is at least b and merge arrivals are at least one tick apart. The last exit is $T_r^* = T_m^*$. □

3.1 When the bounds become exact

Call the instance exit-limited when $T_m^* = D + (N - 1)$ with $K_M \ge 2$. This includes $N \le m$ and $b \le m$.

Theorem 5 (Exact portal counts in the exit-limited regime)**.** *For $K_M \geq 2$ and $r_* \geq 2$, if the optimum meets the exit lower bound, then*

$$L^* = \begin{cases} 2r_*, & Q_0 = 1, \\ r_* + 2, & Q_0 \geq 2. \end{cases}$$

Proof. All exits must occur at $D, D+1, \ldots, D+N-1$. The first r_* require distinct counters because $b \geq r_*$. The first two service starts must be no later than times 2 and 3. A single entry A cell cannot supply both: with its unit receiving and sending rates, its earliest second job reaches a different counter's B cell only at time 4. Therefore at least two A cells are needed, giving the connectivity bound $L \geq r_* + 2$.

If $Q_0 = 1$, the first person must leave the source at time 0 and take a direct two-move approach. Inductively the ith of the first r_* exiting people must leave at time $i-1$ and also take a direct approach: earlier departure slots are already occupied and any extra approach move misses its deadline. These distinct counters therefore require r_* different direct entry/forward pairs. The direct construction matches $2r_*$. □

There is a useful further bound outside the exit-limited regime. Suppose $b > m$, $N > m$, and $K_M \geq 2$, so $r_* = m$. Write

$$N = qm + h, \qquad q \geq 1, \qquad 1 \leq h \leq m.$$

Proposition 6 (A finite-population prefix constraint)**.** *If $Q_0 = 1$, then $L^* \geq m + h$. Consequently, if m divides N, $L^* = 2m$. If $Q_0 \geq 2$ and $h \geq 2$, then $L^* = m + 2$.*

Proof. The deadline is $D + qb + h - 1$. Since $h - 1 < b$, no counter can produce more than $q + 1$ exiting jobs. At least h counters must produce $q + 1$. Their final outputs cannot precede $D + qb$; exactly h output slots remain. Thus these h outputs occur consecutively, forcing their corresponding first service starts no later than $2, 3, \ldots, h + 1$. The source-prefix argument above gives at least h distinct A cells when $Q_0 = 1$, hence $m + h$ portals. With $Q_0 \geq 2$ and $h \geq 2$, the first two forced starts require two A cells; the two-chain construction attains $m + 2$. □

Proposition 7 (A service-limited regime with exact one-feeder sparsity)**.** *If $m \geq 2$, $K_M \geq 2$, $Q_0 \geq 1$, $b \geq 2m - 1$, and $N = qm + 1$ with $q \geq 1$, then $L^* = m + 1$.*

Proof. Use the one-feeder chain. In round k, dispatch counter $h + 1$'s job at $kb + h$ and start it at $2 + kb + 2h$. Take q complete rounds and one final job at the root. The last job from one full round exits at offset $2m - 2$, while the next root's exit is at offset b. The assumption $b \geq 2m - 1$ gives the required one-tick merge separation. All other finite-cell checks are those of the chain construction. The final root exits at $D + qb$, exactly the optimum; connectivity supplies the matching lower bound. □

Population size matters. For $m = 3, p = 4, K_M = 2$, $N = 4$ has $L^* = 4$: service starts at times 2 at the central counter, 4 and 5 at its neighbors, and 6 at the central counter. Exits are 9, 11, 12, 13, reaching $T^* = 13$. The same m, p, K_M with $N = 12$ has $L^* = 5$ when $Q_0 \geq 2$. A formula for portal count must therefore retain N as an input.

m	p	N	K_M	Q_0	T^*	L^*
3	3	12	1	1	30	3
3	3	12	2	2	19	5
3	3	12	2	1	19	6
3	4	4	2	1	13	4
3	4	12	2	2	23	5
4	4	9	2	2	17	6
4	4	9	2	1	17	8
4	5	6	2	1	16	6
4	8	12	2	1	32	8
4	8	12	2	2	32	6

4 Heterogeneous service and finite-graph synthesis

4.1 What remains simple without movement

With instantaneous access, no blocking, and no exit bottleneck, identical jobs on counters with durations p_j have makespan

$$\min\left\{T:\ \sum_{j=1}^{m}\lfloor T/p_j\rfloor \geq N\right\}.$$

This belongs to classical uniform parallel-machine scheduling [1]. It does not choose a physically best queue shape because the stripped model assigns no cost or delay to shape.

4.2 A safe bound with movement

For the direct-row family define $D_j = p_j + 5$ and $b_j = \max(p_j, 2)$ under strict receiving. Every design satisfies

$$N \leq \sum_j U_j(T), \qquad U_j(T) = \max\left\{0, 1 + \left\lfloor \frac{T - D_j}{b_j} \right\rfloor\right\}.$$

For a stronger output-aware lower bound, sort the multiset

$$\{D_j + kb_j :\ j = 1, \ldots, m,\ k = 0, \ldots, N-1\}$$

as $a_1 \leq a_2 \leq \cdots$. Set $\ell_1 = a_1$ and $\ell_i = \max\{a_i, \ell_{i-1} + \delta\}$. Then $T^* \geq \ell_N$. Each counter's kth exit is bounded below by its corresponding stream element, and ordered outputs also obey merge spacing. This bound relaxes the source and approach network, so equality is not guaranteed.

(p_1, p_2, p_3)	K_M	Q_0	Lower bound	T^*	Min. portals
$(1, 3, 5)$	2	3	18	18	5
$(1, 3, 5)$	2	1	18	19	5
$(2, 3, 5)$	2	3	18	18	6
$(2, 3, 5)$	2	1	18	20	4
$(1, 4, 7)$	2	3	19	19	6
$(1, 4, 7)$	2	1	19	20	4
$(2, 5, 8)$	2	3	21	21	5
$(2, 5, 8)$	2	1	21	22	4
$(1, 3, 5)$	1	3	28	28	2
$(2, 3, 5)$	1	3	29	29	2

All rows use $N = 12$. The second and fourth rows disprove treating the sorted-stream relaxation as a general heterogeneous closed form. With $K_M = 1$, the fastest counter alone suffices in the two displayed cases: the exit's alternating movement cycle dominates.

4.3 The general model and its exactness

The supplied program accepts any finite candidate graph whose ordinary cells are partitioned into pre-service and post-service cells. Counters connect the two parts; the source belongs to the first and the exit to the second. The solver chooses cell-build variables h_v, physical-portal variables y_e, and counter-build or placement variables w_j. Ordinary flow cannot cross from the first part to the second. Thus every exiting person must undergo service exactly once.

For a deadline H, the principal integer variables are occupancies $n_v(t)$, movement counts $f_e(t)$, service starts $s_j(t)$, releases $r_j(t)$, and binary counter occupancies $o_j(t)$. In addition to cell balances and sending/receiving constraints, the model imposes

$$o_j(t+1) = o_j(t) + s_j(t) - r_j(t), \qquad 0 \le o_j(t) \le w_j, \tag{5}$$

$$\sum_{u=0}^{t} r_j(u) \le \sum_{u=0}^{t-p_j} s_j(u), \tag{6}$$

$$n_v(t) \le K_v h_v, \qquad \sum_{e \text{ on portal } g} f_e(t) \le Q_g y_g, \tag{7}$$

$$n_{X_1}(H) = N. \tag{8}$$

An empty sum is zero. The release inequality and one-person occupancy enforce nonpreemptive service and permit blocking after completion. Every movement consumes sending capacity and receiving space. Build variables also activate their required endpoints.

For alternative locations of a physical counter, the sum of its placement variables is at most one. The sum of counter variables at a candidate site is also at most one. These constraints allow static counter placement to be optimized without duplicating a physical counter. Longer corridors can be represented by additional cells; non-unit rates are direct model inputs.

Proposition 8 (Finite-model completeness)**.** *For a fixed integer horizon, the formulation is feasible if and only if the stated discrete physical model admits a layout and schedule meeting that horizon and the resource bounds. Therefore minimizing the feasible horizon finds a global optimum over the supplied candidate cells, portals, and placement alternatives.*

Proof. A physical schedule supplies integer occupancies, flows, starts, releases, and build decisions satisfying every constraint. Conversely, replay integer flows using only people present at the beginning of each tick. Sending bounds prevent using fresh arrivals in the same tick; receiving bounds preserve storage. At a counter, the cumulative eligibility inequality and single occupancy match each release to a sufficiently old service start. Inductively all moves and service events are executable. Conservation and the phase partition imply that all N people reaching the exit have each been served exactly once. Counter and site groups enforce the specified placement restrictions. □

This is an established time-indexed integer-optimization technique, not a claimed new algorithmic paradigm. A state-space breadth-first search also provides an exact finite method. For a fixed constructed graph, a safe state-count bound is

$$\prod_{v}(K_v + 1)\prod_{j}(p_j + 2),$$

using cell populations and capped remaining service times. Conservation makes the actual count smaller. Removing repeated states from a successful trajectory yields a finite shortest schedule.

The program also supplies automatic horizon bracketing. If any design is feasible, it contains a source-to-counter-to-exit path. Sending jobs one at a time along a simple such path gives

$$H_{\max} = N\big(|V| - 1 + \max_{j} p_j\big).$$

Nonnegative construction and area costs ensure that the path's subdesign remains within any budgets met by the original design. Deadline feasibility is monotone. Bracketing followed by integer binary search is therefore complete for this model if the underlying solves finish. A time limit is reported as unresolved, never as proof of infeasibility or optimality.

4.4 A discovered spatial topology

We also leave the fixed row architecture. Consider two 3×2 cell grids, one before service and one after it, with a source entering the middle-left approach cell and an exit reachable from the middle-right departure cell. Every grid interface is optional and bidirectional with shared unit capacity. Each cell stores two people. Counter service times at top, middle, and bottom sites are $(2, 4, 3)$ and $N = 8$.

The optimum is **17 ticks**, with eight selected corridors. At that deadline, the minimum illustrative construction cost is 2,200 and the additional area is $28\,\mathrm{m}^2$. The selected network branches before service and merges afterward; neither branch structure was prescribed. Horizon 16 is solver-certified infeasible. The selection is global over all 2^{16} portal subsets, optional cells, and counter-use choices, although branch-and-bound does not explicitly visit every subset.

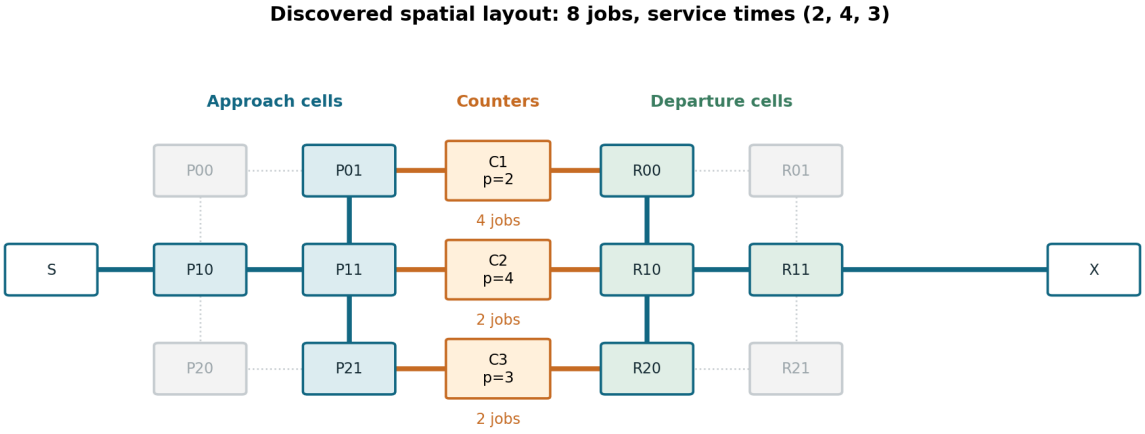


Figure 1: The optimizer chooses both approach and departure corridors. The three candidate service sites are fixed in this figure. The source and exit are existing facilities.

Joint counter placement. If the same three physical counters may be placed at any of the three service sites, the optimum improves to 16 ticks. The joint solver selects the fastest counter in the middle. Checking all six assignments independently confirms the result:

Service durations at (top, middle, bottom)	Optimal clearance
$(2, 4, 3)$	17
$(2, 3, 4)$	17
$(4, 2, 3)$	16
$(4, 3, 2)$	17
$(3, 2, 4)$	16
$(3, 4, 2)$	17

Each permutation is itself optimized over all candidate corridors and cells. These examples demonstrate finite spatial synthesis, including discrete placement; they do not establish an optimal continuous building geometry.

5 A physical construction and area budget

Use the row family with $N = 12$, $(p_1, p_2, p_3) = (1, 3, 5)$, $K_M = 2$, and $Q_0 = 3$. To distinguish actual infrastructure from a portal count, assign the following *illustrative* prices and areas:

- Each storage place costs 100 units and occupies $1\,\mathrm{m}^2$. Thus each A or B cell costs 200 and each E cell costs 100.
- Counters 1, 2, and 3 cost 900, 600, and 450 units, respectively, and each occupies $4\,\mathrm{m}^2$.
- Each constructed portal costs 40 units. Its opening area is included in its adjacent cells to avoid double counting.

These are transparent design inputs, not market quotations or calibrated pedestrian-density recommendations. The source, common merge, and fixed service/departure connections have zero incremental construction cost in this instance. They are excluded from the incremental budgets; the generic graph model can price those links as well. If the same storage-area assumption is used for the source and merge, add a common $14\,\mathrm{m}^2$ to every area in the table.

The model imposes both

$$\sum_v c_v h_v + \sum_j c_j w_j + \sum_e c_e y_e \le B, \qquad \sum_v a_v h_v + \sum_j a_j w_j + \sum_e a_e y_e \le A.$$

Define $C(H)$ as minimum construction cost for clearance by H. The full staircase is:

Budget B	Optimal time	Example area	Minimum-cost design at that time
$B < 1030$	Infeasible	n/a	No complete service route can be built
$1030 \le B < 1180$	65	9	Counter 3, one direct lane
$1180 \le B < 1480$	41	9	Counter 2, one direct lane
$1480 \le B < 2420$	28	9	Counter 1, one direct lane
$2420 \le B < 3450$	20	16	Counters 1 and 2, one shared feeder
$B \ge 3450$	18	25	All three counters, two feeders

Area is in square metres under the illustrative allocation above; it describes a selected cost-optimal design, not a claim that this design alone minimizes area among all cost ties. A separate area-constrained solve proves that $A \le 24$ excludes an 18-tick solution, whereas $A = 25$ admits one.

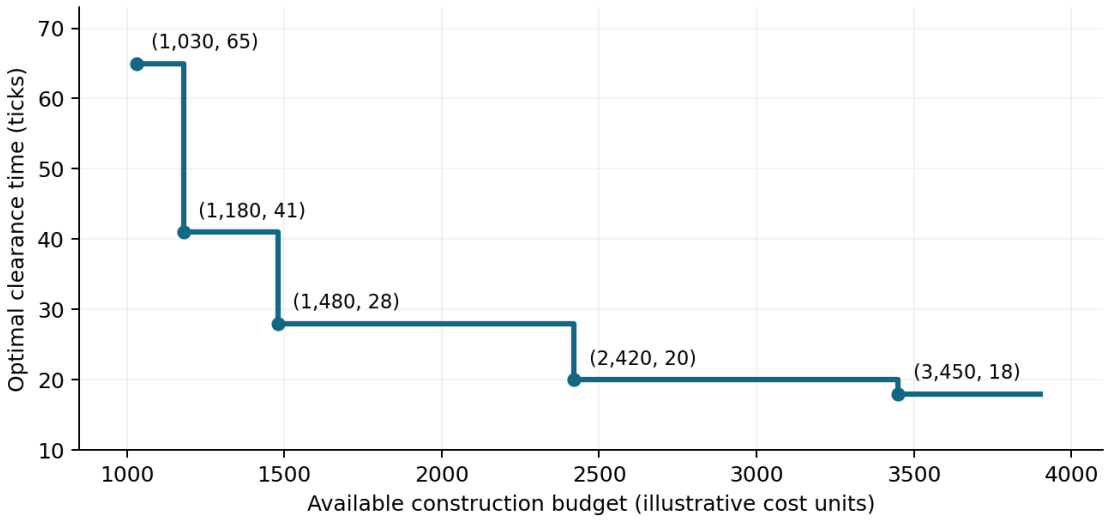


Figure 2: Complete budget-clearance frontier for the specified priced instance.

All deadlines from 18 through 65 are covered. Minimum cost is nonincreasing in the deadline, so equal costs at two endpoints certify the entire interval between them. Recursive endpoint checks require 28 distinct deadline solves. The lower price 1,030 is also elementary: every service route needs a counter costing at least 450, one A, one B, one E, and two approach portals, costing at least $450 + 200 + 200 + 100 + 80$. A direct route to counter 3 attains this and clears in 65 ticks, proving that no further cost reduction appears at a longer horizon.

6 Sensitivity and operating policies

6.1 Receiving-space reuse changes the theorem

Under turnover receiving, retain $O_v(t) \le n_v(t)$ but replace the space bound by

$$I_v(t) \le K_v - n_v(t) + O_v(t).$$

Outgoing people still must be present at the start of the tick, so this is not instantaneous multi-hop movement. It only permits an incoming person to reuse a place vacated during that tick.

Theorem 9 (Uniform services under turnover receiving)**.** *For the same row family and unit movement rates, but turnover receiving,*

$$T^*_{m,\mathrm{turn}} = p + 5 + \max(p, m) \left\lfloor \frac{N-1}{m} \right\rfloor + ((N-1) \bmod m)$$

for every $K_M \ge 1$ and $Q_0 \ge 1$.

Proof. The exit can now accept and discharge in consecutive ticks with one storage place; the one-place departure cells also support successive releases one tick apart. Consequently $\delta = 1$ and $b = p$. The counting lower bounds and direct staggered construction in Theorem 1 apply with these substitutions. Approach cells still have storage two and source admissions remain at most one per tick. □

For $N = 12, m = 3, p = 3, K_M = 1$, the optimum changes from 30 to 19 solely by this convention. This is a decisive model assumption, not a universal claim that a real one-place merge halves capacity. Hardware FIFO documentation explicitly distinguishes simultaneous operations, flag state, and latency [10]; a hardware application must select the corresponding receiving semantics.

6.2 Corridor capacity and merge storage interact

With strict receiving and merge sending/receiving rate Q_M, its long-run movement rate is bounded by

$$\min\{Q_M, K_M/2\}.$$

Indeed $I_M(t) + O_M(t) \le K_M$ in each tick, and over a long clearing stream total input and output differ only by bounded occupancy. This is an asymptotic bound, not an exact finite-population formula. Raising the movement rate without enough storage can therefore leave the bottleneck unchanged.

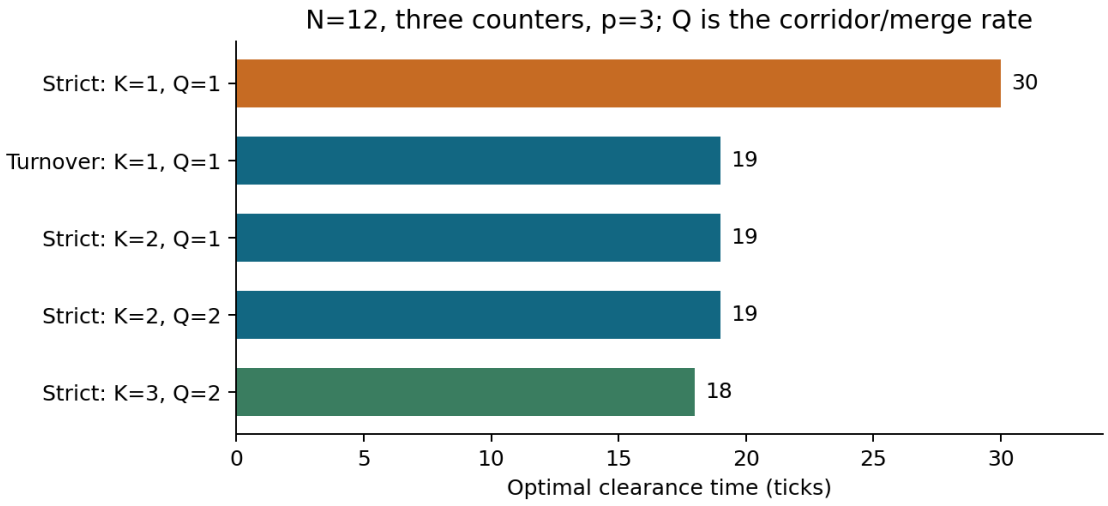


Figure 3: Exact optima when all ordinary corridor, cell-send/receive, and merge rates are changed together. The source rate remains three.

For $p = 1$, strict receiving gives 16 ticks with rate two and $K_M = 2$, and 13 ticks with rate two and $K_M = 3$. The departure cells remain one-place cells, so counter release constraints still matter. No extension of the uniform unit-rate portal formula to arbitrary rates is claimed here; these cases use the general solver.

6.3 Static versus dynamically opened connections

For the original $N = 12, p = 3, K_M = 2, Q_0 = 3$ instance, separate direct lanes attain 19 ticks with six portals, a selected two-feeder design attains 19 with five, and a central pooled feeder with four portals needs 20. For $(p_1, p_2, p_3) = (1, 3, 5)$ the same three fixed designs take 18, 19, and 20, respectively. However, *another* five-portal design attains 18. The identity of a sparse design matters when service times differ.

If constructed portals have no adverse physical effect when unused, a static graph containing the union of all connections used by a dynamic schedule can execute that same schedule. Thus dynamic opening cannot improve the unconstrained optimum or the optimum under a budget on the union of constructed connections. This monotonicity is a property of centrally controlled routing, not a statement about selfish routing equilibria.

Different answers arise under a limit on *simultaneously active* approach portals. In the uniform instance, limits of one, two, and three active portals per tick give optima of 30, 19, and 19. The respective minimum installed counts at those deadlines are four, six, and five. The dynamic model assumes costless instantaneous opening and closing and still charges all installed portals. It does not model moving walls, switching setup time, or physical reconnection work.

6.4 Random durations and decentralized policies

Assign each of the 12 jobs an independent service duration equal to 1 or 5 with equal probability, independent of which counter serves it. This is a stochastic uniform-worker experiment, separate from the heterogeneous deterministic experiments. Each job retains its duration when policies are compared. Controllers observe populations and service completion, but not future durations or remaining durations. All 4,096 possible job-duration assignments are evaluated for each of seven policies, with independent event replay for every trajectory.

The departure merge uses round-robin arbitration. Each eligible counter starts service greedily. At a shared feeder, an eligible own-service action or a downstream branch is selected by local occupancy, with cyclic tie breaking. Source admission is either round robin (RR) or shortest observed local queue/load (JSQ). The latter score includes the receiving lane's approach occupancy, busy-counter indicator, and departure-cell occupancy; it does not divide by a learned service rate or use future samples.

Design	Admission	Q_0	Exact mean	95th percentile	Worst case
Dedicated	RR	3	21.8599	25	27
Two feeders	RR	3	24.7937	29	31
Central pool	RR	3	24.5063	28	32
Dedicated	RR	1	23.5034	27	27
Dedicated	JSQ	1	23.3479	26	27
Two feeders	RR	1	27.6545	33	36
Two feeders	JSQ	1	24.9578	28	31

These are exact expectations and quantiles for the specified equiprobable 4,096 outcomes; no sampling confidence interval is needed. They are policy-specific results, not optimal expected-clearance values.

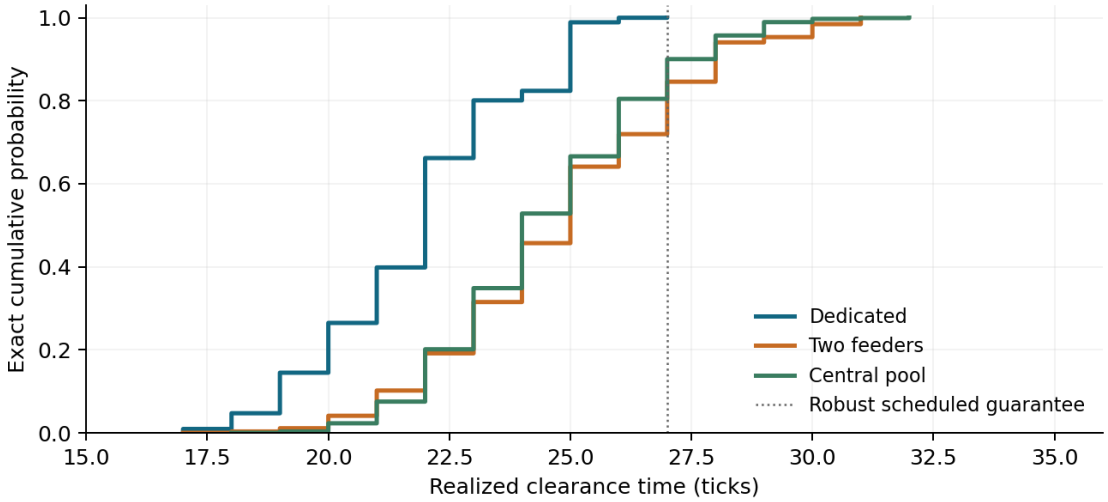


Figure 4: Exact distributions under the tested local policies with source rate three. They do not represent stochastic optimal-control solutions.

The deterministic five-portal design remains capable of 19 ticks for constant duration three, but the tested greedy local controller takes 20. With one source admission per tick, its RR and JSQ policies differ considerably. Consequently an optimal topology theorem does not by itself establish that decentralized operation attains its schedule.

Proposition 10 (An exact robust clearance guarantee)**.** *For $N = 12, m = 3, K_M = 2, Q_0 \geq 2$ and arbitrary job durations in $[1, 5]$, there is a five-portal, nonanticipative schedule with worst-case clearance 27. No design in the row family can have a smaller worst-case clearance.*

Proof. Construct the two-feeder optimal schedule for constant $p = 5$, whose last exit is $10+3\cdot5+2 = 27$. Reserve the same starts and releases for every realization, holding a completed job until its reserved release. All durations are at most five, so the schedule remains feasible without anticipating any duration. The all-five realization belongs to the uncertainty set and has optimum 27 by Theorem 1, giving the lower bound. □

This is a worst-case result. It is not an assertion that the fixed robust schedule minimizes expected clearance. Substituting the mean duration three into the deterministic theorem gives 19, which is not the expected completion time under these stochastic policies.

7 Position relative to established research

Tradition	Established content	Relationship to this work
Deterministic production lines and scheduling	Finite-buffer production lines admit max-plus timing models; instantaneous identical-job allocation to unequal-speed machines is classical [3, 1].	For $m = 1$, our formula is initial latency plus $(N-1)\max(p, 2)$, the deterministic pipeline form. Removing transport and blocking recovers parallel-machine scheduling. The present sparse construction adds physical attainability in a particular parallel-service layout; it does not improve the general scheduling algorithms.
Max-plus algebra	Timed-event recurrences, cycle times, and finite-buffer representations are standard [2, 3].	The two-lag recurrence above is a specialization of max-plus linear dynamics. The closed-form algebra and bottleneck cycle mean are not the novelty claim. Portal-minimal implementations are the additional structural issue.
Finite-buffer queueing networks	Topology, buffers, general service distributions, and blocking have long been studied. Smith, Cruz and van Woensel use approximation and search for buffer allocation in series, merge, and split systems [4].	Our finite-batch, centrally scheduled deadline objective differs from their open-network performance optimization. Exactness for the stated discrete special case does not dominate a more general stochastic approximation model.
Deterministic tandem queues	Yang and Seo derive stationary waiting-time moments, tails, and blocking probabilities for a constant-processing-time, buffer-sharing tandem system driven by Poisson arrivals [5].	The present theorem concerns finite-population clearance through alternative parallel counters. It neither specializes their stationary distribution formulas nor strengthens their probability bounds.
Quickest flows	Fixed-network flows over time, time expansions, and quickest-flow algorithms are established; Fleischer and Skutella provide approximation results for broader flow variants [6].	Our objective is a quickest-clearance objective. Finite storage, atomic nonpreemptive counter occupancy, and construction decisions introduce different constraints. A transit-time arc alone is not a one-person counter. No improvement over their algorithms is claimed.

Tradition	Established content	Relationship to this work
Spillback and spatial queues	Finite-storage spillback already occurs in traffic and dynamic equilibrium models; spatial walking and guidance can alter comparisons between separate and serpentine queues [7, 8].	Congestion propagation and the fact that pooling can lose are not novel claims. Our contribution is a specified controlled finite-state model with exact special-case bounds and explicit assumptions.
Communication networks	Network calculus already supplies deterministic delay and backlog bounds, using arrival/service curves and min-plus methods [9].	The proposed application is choosing lossless finite-burst infrastructure and routing under blocking. This is complementary to those bounds, not an invention of deterministic networking theory.

Defensible contribution statement. "For a specified finite-cell parallel-service family with strict synchronous receiving and blocking after service, we characterize the minimum number of active counters needed for optimal finite-population clearance, prove sharp minimum-portal counts in several regimes, and construct optimal layouts within one portal of the connectivity lower bound whenever the source can admit two jobs per tick. We also provide a finite candidate-graph synthesis formulation and reproducible heterogeneous, budget-constrained, and receiving-rule experiments."

The general-m portal theorems strengthen the earlier *three-counter result in this project*. We do not assert a strict strengthening of the cited general max-plus, queueing-network, or quickest-flow theorems. The exact sparse-attainment statement is a candidate research contribution; asserting it is the first such result would require a more systematic prior-art review. Literature searched for this note includes deterministic manufacturing flow lines, max-plus finite-buffer systems, topological finite-server queueing-network design, quickest flows, spillback, pedestrian queue layouts, and deterministic network calculus, with primary papers or author/publisher sources checked on 27 October 2026.

8 What this can contribute to networks

The cleanest transfer is to a controlled, lossless finite batch moving through finite buffers and alternative workers. A person becomes a packet or task, a counter a packet processor or accelerator, a waiting cell a FIFO or staging buffer, a portal an interconnect, and the merge a shared output channel. Heterogeneous p_j describe different worker speeds. Area and construction budgets become buffer memory, hardware area, link cost, or a weighted combination supplied by the designer.

Three concrete questions are now executable in the finite model: how few interconnects preserve burst-drain time; whether an additional buffer slot is more valuable than another worker; and which worker placement and admission schedule minimize clearance under an area or cost budget. The robust 27-tick schedule also illustrates a deterministic finite-burst completion guarantee when processing times are bounded.

These are proposed applications rather than validated hardware results. General Internet traffic, packet drops and retransmission, several service stages, multiple destinations, fairness, long-run stability, and decentralized strategic routing are not covered by this single-service, single-exit model. FIFO timing must be matched to the actual implementation, including credit-return or flag latency. The mathematical framework is useful precisely because those receiving and capacity choices can be stated, varied, and checked rather than hidden inside a generic queue label.

9 Verification, reproducibility, and remaining scope

Verification category	Count
Strict row-family constructive schedules	8,960
Additional one-feeder slack constructions	432
Turnover-receiving constructive schedules	3,600
Total constructive parameter cases	12,992
Exhaustively enumerated policy trajectories	28,672
Supplemental seeded Monte Carlo trajectories	14,000
Stored generic feasible traces replayed	195
Stored original-family feasible traces replayed	17
Stored solver-infeasible records	82

The strict construction grid ranges over $m = 1, \ldots, 8$, $p = 1, \ldots, 14$, $N = 1, \ldots, 40$, and merge storage one or two. The additional one-feeder slack checks use $m = 2, \ldots, 10$, six service durations from $2m-1$ through $2m+4$, and eight batch counts. Turnover checks use $m = 1, \ldots, 6$, $p = 1, \ldots, 10$, $N = 1, \ldots, 30$, and merge storage one or two. These checks support implementation correctness; the proofs establish the infinite parameter claims.

The code retains every selected graph, cell and counter occupancy, movement, service start, release, objective value, solver status, and gap for the reported optimization instances. The independent replay checks conservation, permitted physical interfaces, storage, sending/receiving limits, duration eligibility, construction and area accounting, placement exclusivity, and that all N jobs are served and exit. The exhaustive stochastic runs also replay realized durations.

The exact portal count remains bounded rather than fully characterized in parts of the service-limited regime, particularly with one source admission per tick. Heterogeneous services have an exact finite optimization procedure and safe lower bounds here, not a general closed form. The area/cost numbers are illustrative inputs. Stochastic policy evaluation and a robust guarantee are completed; optimal expected-clearance control over all nonanticipative policies is not claimed. These boundaries are part of the result, not exceptions suppressed by the experiments.

Reproduction. Use Python 3.11 or later with NumPy, SciPy, and Matplotlib. The repository's README distinguishes quick replay from complete recomputation and documents the generic JSON graph format. Run `python reproduce.py -verify-only` to check stored traces. A full run rebuilds analytic checks, optimization experiments, exhaustive policy outcomes, figures, and the audit. The PDF was typeset with pdfLaTeX; its editable source is included.

References

[1] M. I. Dessouky, B. J. Lageweg, J. K. Lenstra, and S. L. van de Velde (1990). Scheduling identical jobs on uniform parallel machines. *Statistica Neerlandica* 44(3), 115-123. https://doi.org/10.1111/j.1467-9574.1990.tb01276.x. Institutional record: https://ir.cwi.nl/pub/25002.

[2] F. Baccelli, G. Cohen, G. J. Olsder, and J.-P. Quadrat (1992). *Synchronization and Linearity: An Algebra for Discrete Event Systems*. Wiley. Author publication page: https://www.rocq.inria.fr/metalau/cohen/SED/book-online.html.

[3] A. Seleim and H. ElMaraghy (2014). Max-plus modeling of manufacturing flow lines. *Procedia CIRP* 17, 71-75. https://doi.org/10.1016/j.procir.2014.01.133.

[4] J. MacGregor Smith, F. R. B. Cruz, and T. van Woensel (2010). Topological network design of general, finite, multi-server queueing networks. *European Journal of Operational Research* 201(2), 427-441. https://doi.org/10.1016/j.ejor.2009.03.012. Author manuscript dated 22 November 2008: https://www.est.ufmg.br/ftp/fcruz/pub/bap2.pdf.

[5] D. Yang and D.-W. Seo (2017). Closed-form formulae for moment, tail probability, and blocking probability of waiting time in a buffer-sharing deterministic system. *Operations Research Letters* 45(5), 403-408. https://doi.org/10.1016/j.orl.2017.06.004.

[6] L. Fleischer and M. Skutella (2007). Quickest flows over time. *SIAM Journal on Computing* 36(6), 1600-1630. https://doi.org/10.1137/S0097539703427215. Author copy: https://page.math.tu-berlin.de/~skutella/FleischerSkutella.pdf.

[7] L. Sering and L. Vargas Koch (2019). Nash flows over time with spillback. *Proceedings of SODA*, 935-945. https://doi.org/10.1137/1.9781611975482.57.

[8] D. Yanagisawa (2026). Managing pedestrian queueing systems: Adding a counter or providing guidance? *Intelligent Transportation Engineering*, ICITE 2025 proceedings. Published online 9 January 2026. https://doi.org/10.3233/ATDE251491.

[9] J.-Y. Le Boudec and P. Thiran (2001). *Network Calculus: A Theory of Deterministic Queuing Systems for the Internet*. Springer, LNCS 2050. Author-maintained online edition, version 23 August 2022: https://leboudec.github.io/netcal/latex/netCalBook.pdf.

[10] AMD (2023). *Embedded FIFO Generator LogiCORE IP Product Guide*, PG327, version 1.0, Read Operation. https://docs.amd.com/r/en-US/pg327-emb-fifo-gen/Read-Operation.